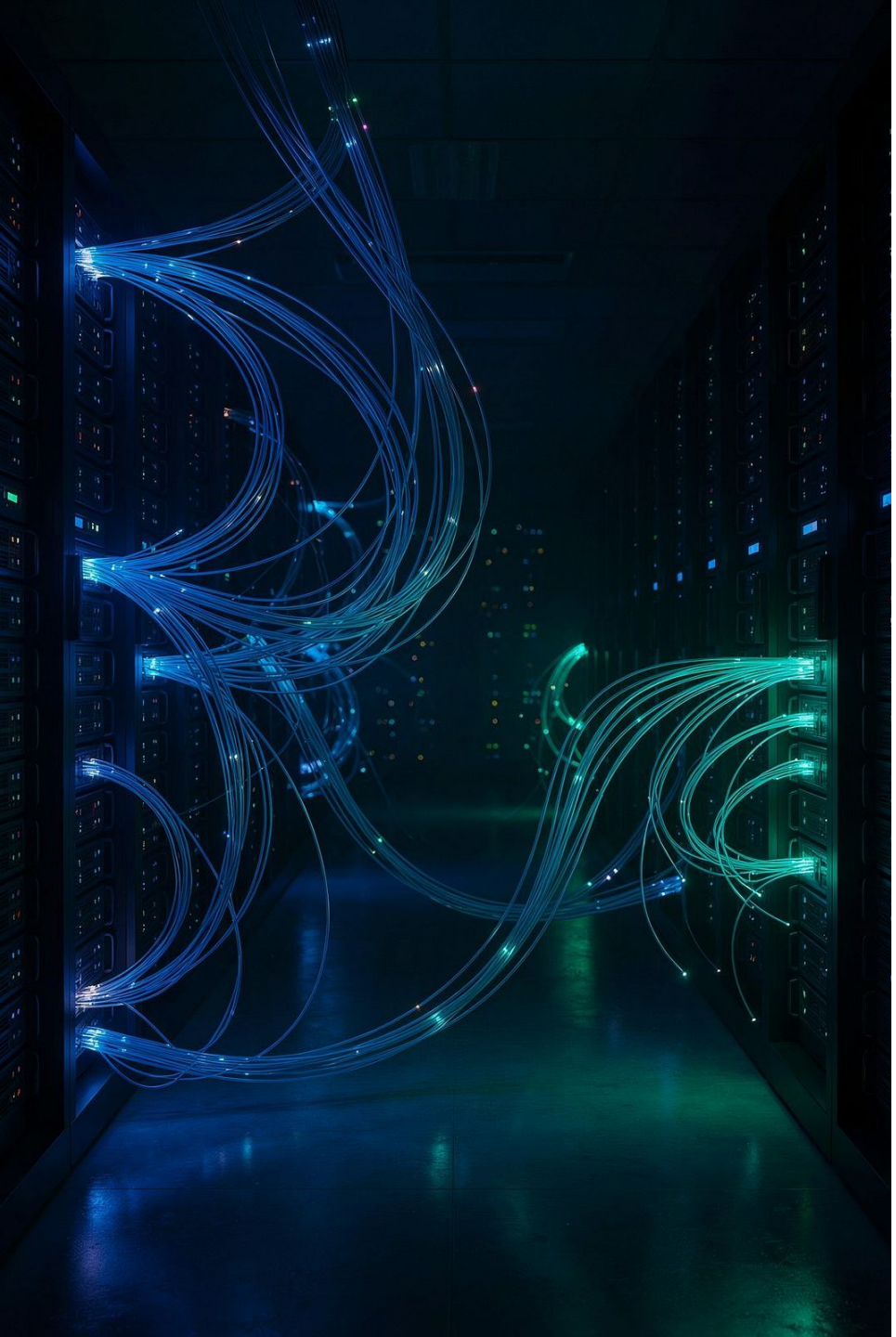


Transformers y LLM

SPRINT 12 · SERIES, LENGUAJE Y GENERATIVA · S12.3

TurisData recibe decenas de mensajes con las mismas preguntas. Un LLM podría contestarlos… si no se inventa nada.

Cómo «lee» un LLM: tokens

1. No lee letras ni palabras completas, sino tokens: trozos frecuentes de texto.
2. Por palabras: vocabularios enormes y no sabe qué hacer con palabras nuevas. Por letras: secuencias larguísimas.
3. BPE: empieza por letras y va fusionando el par más frecuente.

```
pares = contar_pares([list(p) for p in corpus]) par_mas_comun = pares.most_common(1)[0][0]          # Ejercicio 1
```

Tras 12 fusiones aparecen trozos como desayun o limpi: el modelo «descubre» raíces sin saber español (Ejercicio 2: n_tokens_bpe).

Autoatención paso a paso

> Al leer «El hotel es limpio pero ruidoso», para entender ruidoso conviene mirar a hotel. La atención deja que cada palabra decida cuánto mirar a las demás.

1. Tres versiones de cada palabra: Q (qué busco), K (qué ofrezco), V (qué contenido aporto).
2. Puntuaciones $Q \cdot K^T / \sqrt{d}$.
3. Softmax por filas (pesos positivos que suman 1).
4. Salida = pesos · V: cada palabra es una mezcla de las demás según sus pesos.

Atención en código

```
def softmax(m):                                # Ejercicio 3    e = np.exp(m - m.max(axis=1, keepdims=True))    return e / e.sum(axis=1, keepdims=True)  def atencion(X, Wq, Wk, Wv):                                # Ejercicio 4    Q, K, V = X @ Wq, X @ Wk, X @ Wv    pesos = softmax(Q @ K.T / np.sqrt(K.shape[1]))    return pesos @ V, pesos
```

Un Transformer repite esto en muchas capas y «cabezas». Con máscara causal (Ejercicio 5): –1e9 en el triángulo superior (np.triu(k=1)) antes del softmax, para mirar solo hacia atrás.

Generar: logits, softmax y temperatura

1 El modelo da una puntuación (logit) a cada posible siguiente token; el softmax las convierte en probabilidades y se elige una.

2 Temperatura T: reparte las probabilidades.

T baja (0,3)	T alta (2,5)
casi siempre el más probable	más variedad; respuestas estables
más «creatividad» y más riesgo de alucinar	

```
prob_max_T03 = softmax(logits / 0.3).max(); prob_max_T25 = softmax(logits / 2.5).max()  # Ej. 6
```

RAG: que el LLM no invente (cuaderno S12_06)

Un LLM no conoce la política de TurisData y, si no sabe, puede inventar con seguridad. RAG lo evita en 4 pasos:

1. Trocear el documento en fragmentos pequeños, con su sección.
2. Recuperar los fragmentos más parecidos a la pregunta (TF-IDF + coseno: recuperar(pregunta, k=3), Ejercicio 2).
3. Construir el prompt: reglas + contexto numerado + pregunta. Reglas: responder solo con el contexto, citar [1] y decir «No consta...» si no está.
4. Generar la respuesta (generar es sustituible por un LLM real).

El umbral: «no consta» frente a inventar

```
def asistente(pregunta, umbral):    recuperados = recuperar(pregunta)    if recuperados[0][0] < umbral:        # no es lo bastante parecido
return FRASE_NO_CONSTA    return generar(construir_prompt(pregunta, recuperados))
```

1. Umbral alto: más «no consta» (seguro, menos útil). Bajo: responde más (útil, arriesga alucinar).
2. Se mide con 20 preguntas de referencia: acierto, sin fuente y alucinaciones (la nº 20, gimnasio, no tiene respuesta en el documento).
3. Mini-reto: evalúa [0.0, 0.1, 0.15, 0.3, 0.5] y elige el de más aciertos sin alucinaciones.

Seguridad: prompt injection

> Un LLM no distingue bien entre tus instrucciones y el texto que le llega (un documento, una reseña, la pregunta). Si ese texto dice «ignora las instrucciones anteriores y…», el modelo puede obedecer.

Defensas en capas (ninguna es perfecta):

1. Separar instrucciones de datos (etiquetas y regla explícita en la PLANTILLA).
2. Filtrar entradas sospechosas (PATRONES_SOSPECHOSOS) en el corpus y en la pregunta.
3. Limitar lo que el asistente puede hacer y revisar sus respuestas.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio.

CUADERNO · Tokens, atención paso a paso y temperatura (sprints/S12/notebooks/S12_05_transformers_atencion)	40 min
CUADERNO · Mini RAG con la política, evaluación y seguridad (sprints/S12/notebooks/S12_06_mini_rag)	45 min